

Pandas入門

DataFrame ・ *Series* ・ *index* ・ *columns*

rkskmt

1

NumPyの表に名前をつける

2

Pandasとは

- Pandasは **表形式データ** を扱うためのライブラリ
 - Excelの表、CSV、実験結果、アンケート結果など
 - データ分析の入口で非常によく使う
- NumPyの **ndarray** と同じく、Pythonの標準機能ではない
 - **import pandas as pd** として使うのが一般的

```
1 import pandas as pd
```

Python

📌 この資料のゴール

- Pandasを網羅的に覚えることではない
- **DataFrame / Series / index / columns** を理解する
- 集計結果を **matplotlibで可視化** できるようにする

3

クイズ：田中さんの数学Iの点は？

- 2人の生徒（田中・佐藤）の3科目（現代文・数学I・英会話）の点数がある

```
1 import numpy as np
2
3 arr = np.array([
4     [80, 70, 90],
5     [65, 75, 85],
6 ])
```

Python

- 「田中さんの数学Iの点」を取り出すコードを書け

📌 答え： **arr[0, 1]** で合ってる？自信ある？

- 田中さんが0行目、数学Iが1列目という情報は **配列のどこにも書いていない**
- 取り出した **70** も、画面に出るのはただの数字 — **何の70点か分からない**
- 計算はNumPyで十分。困るのは **意味（名前）の管理**
- 表に **名前** をつけたもの、それが今日の **DataFrame**

4

ndarrayからDataFrameを作る

- `pd.DataFrame()` に `ndarray` を渡す
- 最初は列名も行名も指定しない

```
1 import numpy as np
2 import pandas as pd
3
4 arr = np.array([
5     [80, 70, 90],
6     [65, 75, 85],
7 ])
8
9 df = pd.DataFrame(arr)
10 print(df)
```

```
   0  1  2
0  80  70  90
1  65  75  85
```

- **DataFrame** は **表** として表示される
- 左端と上端に **0, 1, 2, ...** の番号が自動でつく

5

indexとcolumnsを確認

- 行の名前を **index**
- 列の名前を **columns** という
- 指定しない場合は、どちらも自動で番号が振られる

```
1 print(df.index)
```

```
RangeIndex(start=0, stop=2, step=1)
```

```
1 print(df.columns)
```

```
RangeIndex(start=0, stop=3, step=1)
```

① 自動でつく番号

- **RangeIndex(start=0, stop=2, step=1)** は「0から1までの行番号」
- **RangeIndex(start=0, stop=3, step=1)** は「0から2までの列番号」
- Pythonの **range()** と同じく、**stop** の値は含まない

6

columnsを指定する

- 列の意味が分かるように **columns=** を指定する
- これだけで表がかなり読みやすくなる

```
1 df = pd.DataFrame(  
2     arr,  
3     columns=["現代文", "数学I", "英会話"]  
4 )  
5  
6 print(df)
```

	現代文	数学I	英会話
0	80	70	90
1	65	75	85

- もう **df[1]** ではなく、**df["数学I"]** として扱える
- 「何列目か」ではなく **何の列か** で読める

7

indexも指定する

- 行の意味が分かるように **index=** も指定する
- この例では、行名を生徒名にする

```
1 df = pd.DataFrame(  
2     arr,  
3     columns=["現代文", "数学I", "英会話"],  
4     index=["田中", "佐藤"]  
5 )  
6  
7 print(df)
```

	現代文	数学I	英会話
田中	80	70	90
佐藤	65	75	85

💡 DataFrameの正体

```
1 DataFrame = データ + 行名(index) + 列名(columns)
```

8

名前があると何がうれしい？



- `df["数学I"]` と書くと、数学Iの列だけ取り出せる
- `df.mean()` と書くと、列ごとの平均が計算できる
- 出力にも列名が残るので、結果を読み間違えにくい

```
1 print(df.mean())
```

```
現代文    72.5
数学I     72.5
英会話    87.5
dtype: float64
```

- NumPyの **axis=0** より読みやすい場面が多い
- データ分析では **名前付きの表** が強い

9

Seriesを理解する

10

Seriesを取り出す

- `df["列名"]` で1列だけ取り出せる
- 取り出した1列は **DataFrame** ではなく **Series**

```
1 math = df["数学I"]
2 print(math)
```

```
田中    70
佐藤    75
Name: 数学I, dtype: int64
```

```
1 print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
1 print(type(math))
```

```
<class 'pandas.core.series.Series'>
```

11

DataFrameとSeries

- DataFrameは **表**
- Seriesは **1列**
- Seriesにも **index** がある

```
1 DataFrame = 表
2 Series    = 1列
```

```
1 print(math.index)
```

Python

Index(['田中', '佐藤'], dtype='object')

```
1 print(math.values)
```

Python

[70 75]

- **math.index** は生徒名
- **math.values** は点数だけを取り出したNumPy配列

Seriesは「値」と「名前」のセット

- NumPy配列は値だけ
- Seriesは値に **index** がついている

見方	中身
math.index	["田中", "佐藤"]
math.values	[70, 75]

```
1 print(math.mean())
```

Python

72.5

- 1列だけでも、平均・最大・最小などの計算ができる
- **DataFrame** から列を取り出すと、集計しやすい

列名で取り出す練習

- 現代文の列を取り出す
- 英会話の列を取り出す
- それぞれの平均を計算する

```
1 japanese = df["現代文"]
2 english = df["英会話"]
3
4 print(japanese.mean())
5 print(english.mean())
```

Python

72.5
87.5

💡 まずは列単位で考える

- Pandasでは「列を取り出す → 計算する」が基本
- 1列を取り出すと **Series**
- 複数列を取り出すと **DataFrame**

14

CSVを読み込む

15

CSVとは

- CSVは表をテキストで保存する形式
 - **C**omma **S**eparated **V**alues
 - カンマで値を区切る
- ExcelやGoogleスプレッドシートからも書き出せる

```
1 名前,学科,学年,現代文,数学I,英会話,物理,欠席日数
2 田中,情報,1,80,70,90,85,2
3 佐藤,機械,1,65,75,85,72,4
4 鈴木,情報,2,88,91,84,90,1
```

- 1行目は列名として扱われることが多い
- 列が増えるほど、Pandasで扱う意味が出てくる
- Pandasでは **pd.read_csv()** で読み込む

16

CSVを読み込む

- 授業用CSVは [data/students.csv](#) からダウンロードできる
- 自分の作業フォルダに **data** フォルダを作る
- その中に **students.csv** を置く
- Pythonからは **data/students.csv** というファイルパスで読む

```
1 import pandas as pd
2
3 csv_path = "data/students.csv"
4 df = pd.read_csv(csv_path)
5 print(df.head())
```

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	情報	1	80	70	90	85	2
1	佐藤	機械	1	65	75	85	72	4
2	鈴木	情報	2	88	91	84	90	1
3	高橋	電気	2	72	68	76	80	3
4	伊藤	情報	1	58	57	62	65	6

- CSVの1行目が **columns** になる
- **index** は指定しなければ自動で番号が振られる

読み込んだ表の形を確認

- データを読み込んだら、まず **中身と形** を確認する
- **head()** は先頭数行を表示する
- **shape** は **(行数, 列数)**

```
1 print(df.head())
```

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	情報	1	80	70	90	85	2
1	佐藤	機械	1	65	75	85	72	4
2	鈴木	情報	2	88	91	84	90	1
3	高橋	電気	2	72	68	76	80	3
4	伊藤	情報	1	58	57	62	65	6

```
1 print(df.shape)
```

(20, 8)

```
1 print(df.columns)
```

Index(['名前', '学科', '学年', '現代文', '数学I', '英会話', '物理', '欠席日数'], dtype='object')

```
1 print(df.index)
```

RangeIndex(start=0, stop=20, step=1)

名前をindexにする読み込み

- `index_col="名前"` とすると、名前列をindexにできる
- 「行名として使いたい列」があるときに使う

```
1 df_name = pd.read_csv(csv_path, index_col="名前")
2 print(df_name.head())
```

Python

	学科	学年	現代文	数学I	英会話	物理	欠席日数
名前							
田中	情報	1	80	70	90	85	2
佐藤	機械	1	65	75	85	72	4
鈴木	情報	2	88	91	84	90	1
高橋	電気	2	72	68	76	80	3
伊藤	情報	1	58	57	62	65	6

- ただし、最初は自動番号のindexでも問題ない
- 「indexとは行名である」と理解できれば十分

📌 今回の方針

- `loc` / `iloc` の詳細は深追いしない
- まずは **列選択・条件抽出・集計** に慣れる

単一列を選択する

- `df["列名"]` で1列を取り出す
- 返ってくるものは **Series**

```
1 math = df["数学I"]
2 print(math)
```

Python

```
0      70
1      75
2      91
3      68
4      57
5      88
6      73
7      65
8      54
9      95
10     86
11     72
12     80
13     78
14     84
15     61
16     89
17     66
18     82
19     93
Name: 数学I, dtype: int64
```

```
1 print(type(math))
```

Python

```
<class 'pandas.core.series.Series'>
```

- 1列だけ取り出したら **Series**
- 集計の入口としてよく使う

複数列を選択する

- `df[["列名1", "列名2"]]` で複数列を取り出す
- 列名のリストを渡すので、角括弧が2重になる
- 返ってくるものは **DataFrame**

```
1 print(df[["名前", "学科", "数学I", "英会話"]])
```

	名前	学科	数学I	英会話
0	田中	情報	70	90
1	佐藤	機械	75	85
2	鈴木	情報	91	84
3	高橋	電気	68	76
4	伊藤	情報	57	62
5	渡辺	機械	88	78
6	山本	電気	73	71
7	中村	情報	65	82
8	小林	機械	54	60
9	加藤	電気	95	88
10	吉田	情報	86	91
11	山田	機械	72	66
12	佐々木	電気	80	69
13	山口	情報	78	75
14	松本	機械	84	73
15	井上	電気	61	72
16	木村	情報	89	94
17	林	機械	66	58
18	清水	電気	82	80
19	斎藤	情報	93	85

```
1 print(type(df[["名前", "学科", "数学I", "英会話"]]))
```

<class 'pandas.core.frame.DataFrame'>

⚠ 角括弧を重ねる理由

```
1 df["数学I"]           # 列名を1つ渡す
2 df[["名前", "学科", "数学I"]] # 列名のリストを渡す
```

条件抽出

- 条件式を書くと、各行について **True** / **False** が返る
- その **True** の行だけを取り出せる

```
1 print(df["数学I"] >= 80)
```

```
0    False
1    False
2     True
3    False
4    False
5     True
6    False
7    False
8    False
9     True
10    True
11   False
12    True
13   False
14    True
15   False
16    True
17   False
18    True
19    True
Name: 数学I, dtype: bool
```

```
1 print(df[df["数学I"] >= 80])
```

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
2	鈴木	情報	2	88	91	84	90	1
5	渡辺	機械	3	84	88	78	86	2
9	加藤	電気	3	90	95	88	92	0
10	吉田	情報	3	83	86	91	89	1
12	佐々木	電気	1	74	80	69	83	2
14	松本	機械	3	81	84	73	88	1
16	木村	情報	1	92	89	94	91	0
18	清水	電気	3	86	82	80	87	2
19	斎藤	情報	3	77	93	85	90	1

- **数学I** が80点以上の行だけが残る
- 条件式をそのまま **df[...]** の中に入れる

文字列の条件抽出

- 数値だけでなく、文字列でも条件を作れる
- `==` は「等しいかどうか」を調べる演算子

```
1 print(df[df["学科"] == "情報"])
```

Python

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
0	田中	情報	1	80	70	90	85	2
2	鈴木	情報	2	88	91	84	90	1
4	伊藤	情報	1	58	57	62	65	6
7	中村	情報	2	76	65	82	74	3
10	吉田	情報	3	83	86	91	89	1
13	山口	情報	2	62	78	75	70	5
16	木村	情報	1	92	89	94	91	0
19	斎藤	情報	3	77	93	85	90	1

- **学科** が **"情報"** の行だけが残る
- アンケートや名簿データではよく使う

24

複数条件で抽出する

- 複数条件は `&` でつなぐ
- それぞれの条件を丸括弧で囲む

```
1 print(df[
2     (df["学科"] == "情報")
3     & (df["数学I"] >= 80)
4 ])
```

Python

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数
2	鈴木	情報	2	88	91	84	90	1
10	吉田	情報	3	83	86	91	89	1
16	木村	情報	1	92	89	94	91	0
19	斎藤	情報	3	77	93	85	90	1

⚠ **and** ではなく **&**

- Pandasの条件抽出では **and** ではなく **&**
- 条件ごとに丸括弧をつける
- この書き方は最初だけ少し変だが、よく使う

25



- 前ページのコード の **&** を **|** （または） に変えて実行
→ 「両方を満たす行」 → 「どちらか一方でも満たす行」 に変わり、残る行が増える
- さらに条件もいじる（項目ごとに都度実行）

こう変える	変化
>= 80 → < 80	条件が反転し、残る生徒が入れ替わる
末尾に & (df["英会話"] >= 80) を足す	条件が増えて、さらに絞り込まれる
>= 80 → >= 70	基準がゆるみ、残る行が増える

💡 **そもそもここから始めるのが筋**

- まず条件式だけを表示：**(df["学科"]=="情報") & (df["数学I"]>=80)**
- 各行が両条件を満たすかの True / False の列が返る
- **df[...]** は、その True の行だけを取り出している

列を追加する

- **df["新しい列名"] = ...** で列を追加できる
- 既存の列を使って、新しい列を計算することが多い

```
1 subjects = ["現代文", "数学I", "英会話", "物理"]
2
3 df["平均点"] = df[subjects].mean(axis=1)
4 df["合否"] = df["平均点"] >= 60
5 df["補正数学I"] = df["数学I"] * 1.1
6
7 print(df.head())
```

Python

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数	平均点	合否	補正数学I
0	田中	情報	1	80	70	90	85	2	81.25	True	77.0
1	佐藤	機械	1	65	75	85	72	4	74.25	True	82.5
2	鈴木	情報	2	88	91	84	90	1	88.25	True	100.1
3	高橋	電気	2	72	68	76	80	3	74.00	True	74.8
4	伊藤	情報	1	58	57	62	65	6	60.50	True	62.7

- **平均点** は4科目の平均
- **合否** は平均点が60点以上かどうか
- **補正数学I** は数学Iを1.1倍した値

① **NumPyと同じ感覚**

- **df["数学I"] * 1.1** は、数学I列の各要素をまとめて計算する
- **mean(axis=1)** は、行ごとに平均を計算する
- for文を書かなくても列全体に計算がかかる

ここから使うCSVデータ

- ここからは **data/students.csv** から読み込んだ **df** を前提にする
- CSVには、名前・学科・学年・4科目・欠席日数が入っている
- まず4科目から **平均点** 列を作る

```
1 csv_path = "data/students.csv"
2 df = pd.read_csv(csv_path)
3
4 subjects = ["現代文", "数学I", "英会話", "物理"]
5 df["平均点"] = df[subjects].mean(axis=1)
6
7 print(df.head())
```

	名前	学科	学年	現代文	数学I	英会話	物理	欠席日数	平均点
0	田中	情報	1	80	70	90	85	2	81.25
1	佐藤	機械	1	65	75	85	72	4	74.25
2	鈴木	情報	2	88	91	84	90	1	88.25
3	高橋	電気	2	72	68	76	80	3	74.00
4	伊藤	情報	1	58	57	62	65	6	60.50

i CSVを読んだあとの典型的な流れ

1 CSVを読む → 必要な列を作る → 集計する → 可視化する

平均点列を集計する

- まず **平均点** 列を **Series** として取り出す
- Seriesには平均・最大・最小などのメソッドがある

```
1 scores = df["平均点"]
2
3 print(scores.mean())
4 print(scores.max())
5 print(scores.min())
```

76.7875
91.5
56.75

- mean()** は平均
- max()** は最大
- min()** は最小

学科ごとに平均点を出す

- `groupby("学科")` で学科ごとに分ける
- その後 `["平均点"].mean()` で平均点の平均を出す
- CSVの **学科** 列と、追加した **平均点** 列を使っている

```
1 avg = df.groupby("学科")["平均点"].mean()  
2 print(avg)
```

```
学科  
情報      80.062500  
機械      71.625000  
電気      77.583333  
Name: 平均点, dtype: float64
```

- 学科ごとの平均点が求まる
- 結果は **DataFrame** ではなく **Series**

31

groupbyの結果を確認

- `groupby`の結果も **Series**
- **index** が学科名
- **values** が平均点

```
1 print(type(avg))
```

```
<class 'pandas.core.series.Series'>
```

```
1 print(avg.index)
```

```
Index(['情報', '機械', '電気'], dtype='object', name='学科')
```

```
1 print(avg.values)
```

```
[80.0625    71.625    77.58333333]
```

💡 groupbyの結果

- ```
1 groupby の結果も Series
2 index が「学科」
3 values が「平均点」
```

32

# Tweak : groupby は「分ける × 集計する列 × 集計方法」



- 前ページのコード の3か所を、以下に差し替えて実行  
→ 項目ごとに都度実行すること

| こう変える                                                                                    | 変化                   |
|------------------------------------------------------------------------------------------|----------------------|
| <code>groupby("学科")</code> → <code>groupby("学年")</code>                                  | 「分ける軸」が変わる           |
| <code>["平均点"]</code> → <code>["欠席日数"]</code>                                             | 集計列が変わる              |
| <code>.mean()</code> → <code>.max()</code> / <code>.min()</code> / <code>.count()</code> | 「集計方法」が変わる（最大／最小／件数） |

💡 そもそもここから始めるのが筋

- `df.groupby("学科")` だけを表示して確認
- `df.groupby("学科")["平均点"]` も確認

## locとilocを少しだけ

- `loc` は **index名** で取り出す
- `iloc` は **位置番号** で取り出す
- 今回はこの程度で十分

```
1 print(avg.loc["情報"])
```

Python

80.0625

```
1 print(avg.iloc[0])
```

Python

80.0625

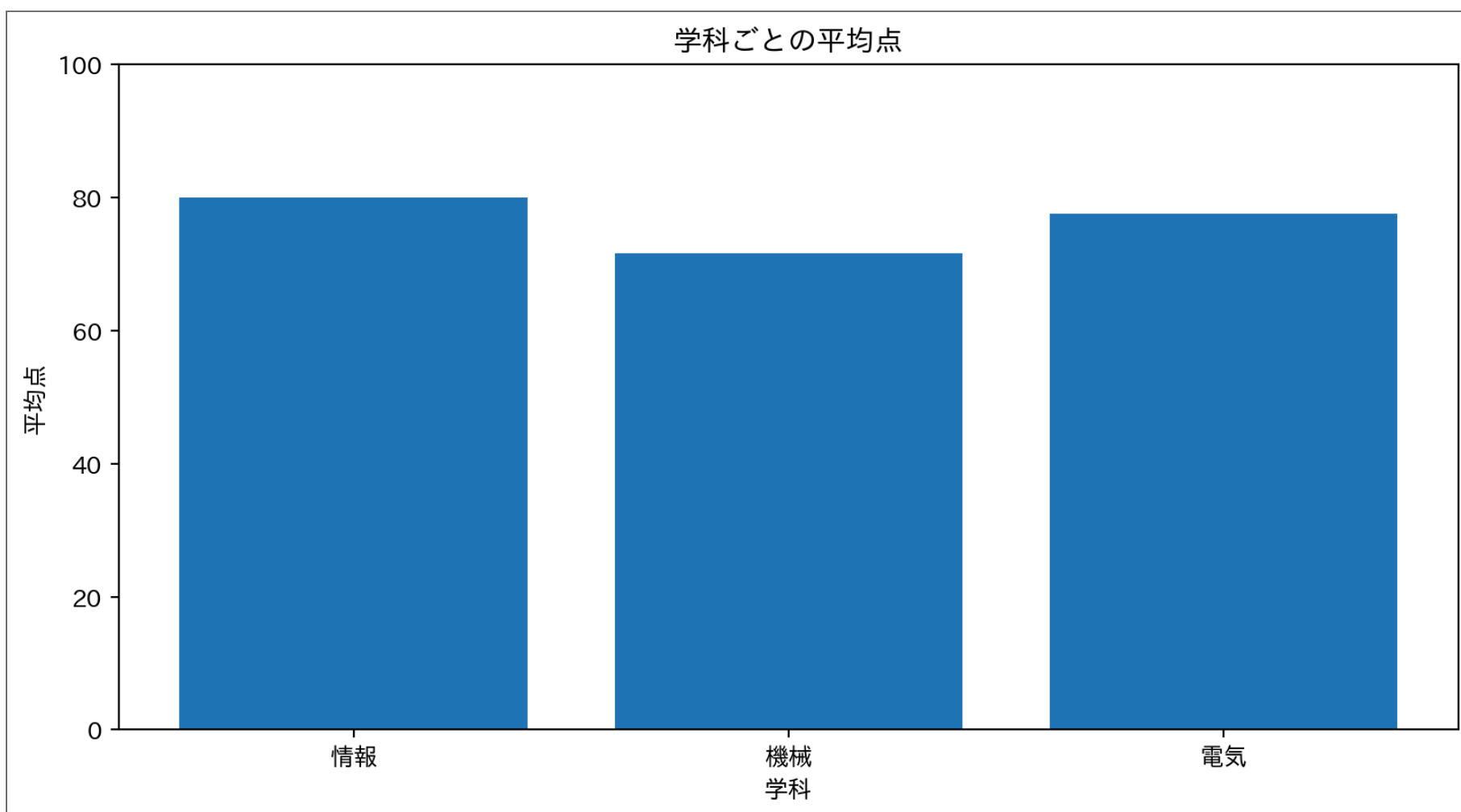
- `avg.loc["情報"]` は「情報」の平均点
- `avg.iloc[0]` は先頭の平均点



## matplotlibで棒グラフにする

- Pandasの **.plot.bar()** は今回は使わない
- **Series** を **index** と **values** に分けて matplotlib に渡す
- ここでは **groupby()** で作った **avg** を可視化する

```
1 import matplotlib.pyplot as plt
2 import japanize_matplotlib
3
4 plt.bar(avg.index, avg.values)
5 plt.xlabel("学科")
6 plt.ylabel("平均点")
7 plt.title("学科ごとの平均点")
8 plt.ylim(0, 100)
9 plt.show()
```



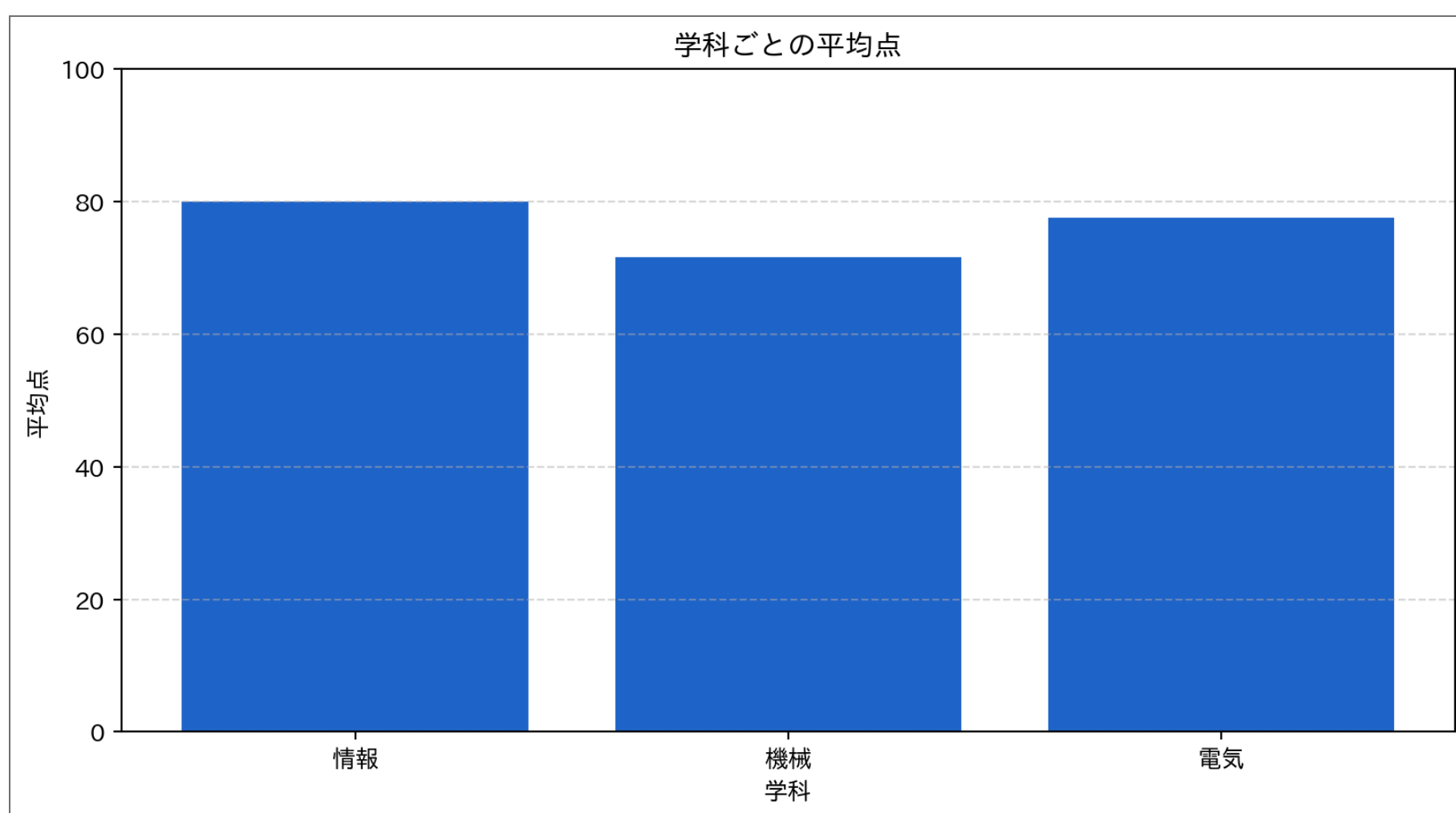
### 💡 Pandasからmatplotlibへ

```
1 Series → index / values → matplotlib
```

## 少し整える

- 色やグリッドを足すと、比較しやすくなる
- 棒グラフでは **ylim(0, 100)** のように、0から見せることが多い

```
1 import matplotlib.pyplot as plt
2 import japanize_matplotlib
3
4 plt.bar(avg.index, avg.values, color="#2266CC")
5 plt.xlabel("学科")
6 plt.ylabel("平均点")
7 plt.title("学科ごとの平均点")
8 plt.ylim(0, 100)
9 plt.grid(axis="y", linestyle="--", alpha=0.5)
10 plt.show()
```



36

## 演習：高得点者を可視化



- **data/students.csv** を読み込む
  - 4科目の平均点を計算する
  - 平均点が80点以上の生徒だけを抽出する
  - 生徒名を横軸、平均点を縦軸にして棒グラフを描く
  - y軸は **0** から **100** にする
- ▶ 解答例を見る

37

# 実データで：明治の東京 vs 21世紀の東京

- 可視化編で使った気象庁データをPandasで読む
  - データ：[tokyo\\_temp\\_annual.csv](#) (data フォルダに置く)
- 「明治時代（～1911年）の平均気温」と「2000年以降の平均気温」 — 条件抽出がそのまま効く

```
1 import pandas as pd
2
3 df_temp = pd.read_csv("data/tokyo_temp_annual.csv")
4
5 meiji = df_temp[df_temp["year"] <= 1911]
6 recent = df_temp[df_temp["year"] >= 2000]
7
8 print(f"明治の東京 : {meiji['temp'].mean():.1f} °C")
9 print(f"21世紀の東京 : {recent['temp'].mean():.1f} °C")
```

明治の東京 : 13.8 °C  
21世紀の東京 : 16.7 °C

- 150行のデータから **2行で答えが出る**
- NumPyなら「何列目が年だっけ？」から始まる作業が、**列名で読み書きできる**

## 💡 CSVを読んだあとの典型的な流れ（再掲）

1 CSVを読む → 必要な列を作る → 絞る・集計する → 可視化する

どんなデータでも、この型は同じ

38

# 伏線：あの山の「日付」を特定するには

- 可視化編の冒頭の謎 — 1年分の閲覧数で **1日だけの大きな山**
- 山の **高さ** は **max()** で分かる。でも知りたいのは **「いつ」**
- 今日の道具がそのまま効く：

```
1 df[df["views"] == df["views"].max()] # 最大値の行を「日付ごと」取り出す
```

- 閲覧数と日付を **表** にしてしまえば、条件抽出1行
- 謎を解く日は近い — 道具はもうほとんど揃っている

39



## まとめ

---

- **DataFrame** は 表  
→ データ + **index** + **columns**
- **Series** は 1列  
→ 値 + **index**
- **pd.read\_csv()** でCSVを読み込める
- 条件抽出は **df[条件]**
- **groupby()** でグループごとの集計ができる
- 集計結果の **index** / **values** を matplotlib に渡して可視化できる

40

## 今回は深追いしないこと

---

- **apply**
- **merge**
- **pivot\_table**
- **MultiIndex**
- 欠損値処理
- **datetime**
- **loc** / **iloc** の詳細

### ① まず必要な型を見失わない

- **DataFrame** なのか
- **Series** なのか
- **index** と **columns** は何なのか

41